Smart Voicemail Router

Team Allocation and Delivery Plan

Document ID: SVR-TEAM-03
Version: 1.2
Status: Proposed pair allocation for team review
Updated: 10 November 2026
Formal project kickoff: 11 November 2026
Competition deadline: Unknown; plan uses relative, gated phases

> **Allocation rule:** Each pair owns a substantial part of the working system. Frontend and deployment remain unassigned at kickoff. After the first integrated fixture run, Hazem will name the implementer(s) before demo-readiness; his own implementation role remains open.

1. How this allocation was chosen

The workstreams follow the system boundaries in SVR-ARCH-02: (1) audio and speech evidence, (2) complaint and customer understanding, and (3) decision, routing, and evaluation. They are proposals based on the architecture, not claims about members' personal skills. The team can adjust the split at kickoff while preserving one shared data contract.

Every pair will independently review the same approved dataset snapshot before the team finalizes labels or evaluation splits. Final classes require team agreement. Pairs can start EDA, fixture-first scaffolding, provider-adapter exploration, and isolated tests before the taxonomy is final.

2. Pair workstreams

Pair	Primary project workstream	Accountabilities and deliverables	Team responsibility
Hazem & Ibthal	**Audio and speech evidence**	Local audio intake/validation; safe preprocessing; STT provider interface and benchmark; transcript confidence/language-mix metadata; audio quality and acoustic features; optional voice-state candidate only if data/model validation supports it; tests and run evidence.	Owns the audio-to-evidence path and its measurable limits.
Lama & Ahmed	**Complaint and customer understanding**	Shared EDA; transcript normalization and Arabic/English telecom glossary; local identifier masking; text-analysis schema; summary and customer request; separate complaint category, complaint urgency, customer-state-from-text, and optional churn risk; suggested opening line; schema/quality tests.	Owns a tested analysis contract grounded in the dataset.
Sira & Aya	**Decision, routing, and evaluation**	Shared EDA; agent skill/availability data contract; explainable fusion and queue priority; agent eligibility/matching and no-match behavior; routing reasons; integration/evaluation runner; metrics, run records, and audit-event checks.	Owns testable routing outcomes and evidence that the full pipeline behaves as intended.

Reserved assignments

- **Frontend implementation:** remains unassigned at kickoff. After the first integrated fixture run, at the end of core work and before demo-readiness, Hazem will name the implementer(s). The dashboard still needs upload, browser recording, a live queue, ticket evidence, and a callback brief. Until the assignment, all pairs should keep API/data contracts stable and use mocks or test clients.
- **Deployment/packaging:** remains unassigned at kickoff. At the same late-core gate, Hazem will name the implementer(s) before demo-readiness. The local run must remain reproducible. If capacity is short, Hazem must reallocate or reduce scope explicitly rather than silently dropping the demo requirement.

3. Shared dataset EDA: first evidence gate

Each pair will independently analyze the **same dataset version/snapshot** and create a short, reproducible EDA note before final taxonomy or evaluation splits are approved. Compare all three notes at the earliest weekly team meeting, correct disagreements, and produce one shared data specification.

The common EDA checklist:

1. Confirm data origin, license/consent, competition permission, and allowed processing locations before opening or copying recordings.
2. Inventory file count, formats, duration, sample rate, channel count, clipping/noise, and unreadable/corrupt samples.
3. Review language mix, dialect/code-switching patterns, and transcript/reference availability.
4. Count missing fields, duplicates, near-duplicates, and repeated speakers/sources where identifiable.
5. If labels exist, inspect definitions, annotator consistency, class counts, ambiguous examples, and possible leakage. If labels do not exist, propose an annotation protocol but do not finalize labels independently.
6. Assess identifier and voice privacy risks; set a local-only boundary unless the team explicitly approves an external-provider experiment.
7. Propose a speaker/source-aware data split where feasible, keeping an evaluation set separate from tuning.
8. Record commands/notebook version, dataset snapshot/checksum if permitted, assumptions, limitations, and recommended next decisions.

Exit gate: the team compares all three EDA notes and approves one data dictionary, label/annotation guide, privacy and retention rules, and evaluation split plan. Class labels and accuracy claims require this shared approval.

4. Cross-pair contracts

Producer	Contract delivered	Consumer	Boundary rule
Hazem & Ibthal	Audio evidence: local audio reference, transcript and STT confidence/language metadata, audio quality and acoustic evidence; optional voice-state candidate kept separate.	Lama & Ahmed; Sira & Aya	Audio output does not decide complaint urgency or queue route. Raw recordings stay within the approved data boundary.
Lama & Ahmed	Structured text analysis with summary, approved complaint category, complaint urgency, customer-state-from-text, customer request, optional churn risk, review flag, and schema/label version.	Sira & Aya; future UI assignment	Enum values come only from the agreed data specification; unknown/unclear values remain reviewable.
Sira & Aya	Explainable triage decision with priority, assignment/unassigned status, route reason, contributing rule/config version, and evaluation result.	Future UI/deployment assignment; team review	Decision logic consumes separate inputs; no emotion/urgency conflation and no hidden hard-filter bypass.

Pairs can start EDA, fixture-first scaffolding, provider adapters, and isolated tests in parallel. Before production integration, agree stable field names, enum/null/unknown behavior, error states, and versioning. The final label values and production schema are frozen only after the shared EDA/data-spec gate. Interface changes require a short written decision and coordinated updates to tests and consumers.

5. Relative delivery phases

The competition deadline and weekly time availability are unknown, so the plan uses ordered phases and exit gates instead of calendar estimates. Once the deadline and rubric are confirmed, the team can reprioritize phases without changing their evidence requirements.

Phase	Main work	Exit gate
Phase 0: Formal kickoff	Confirm pairs, shared working rules, competition restrictions, dataset access request, and privacy boundary. Formal kickoff is 11 November 2026.	Pair ownership, data access constraints, and immediate blockers are visible.
Phase 1: Independent EDA	Each pair runs the same checklist on the same authorized dataset snapshot; prepare three separate notes. Fixture-first setup and isolated tests can proceed in parallel.	Earliest weekly meeting compares the notes; discrepancies and risks are recorded.
Phase 2: Shared data and interface freeze	Agree data dictionary, complaint-category/urgency/customer-state definitions, optional churn field, annotation guide, data split, JSON/API schema, privacy rules, and baseline metrics.	One approved, versioned production contract exists; provisional work can be reconciled against it.
Phase 3: Pair-owned core implementation	Build the three workstreams against fixtures and then the approved contract; include tests and traceable experiment records.	Each pair demonstrates its inputs/outputs and records known limits.
Phase 4: Integration and measurement	Join audio, text analysis, fusion, routing, storage, and evaluation; run fixed-data/config benchmarks; report actual metrics. Hazem reviews contribution/capacity and names UI/deployment implementer(s) at the end of this core phase.	One reproducible end-to-end fixture/data run; named UI/deployment owners and a feasible demo scope before demo-readiness.
Phase 5: Demonstration and handoff	Complete the dashboard, including upload and browser recording; finish local packaging/run instructions; polish the demo, backup, and results summary. Keep the final document set editable as Markdown and deliver clean PDF copies.	Demo runs cleanly under declared conditions; no unmeasured claim is presented; PRD/evaluation results can be shared as Markdown and PDF.

The phases follow their exit gates rather than calendar estimates. Revisit their order at the weekly meeting and record the reason. A blocked data/privacy gate does not permit model work on unapproved material.

6. Weekly working cadence and future repository flow

- Each pair provides a weekly update and pushes validated progress to its own branch once the repository and branches are set up in the later authorized phase. Smaller, reviewable pushes are welcome; no fixed weekly hour quota is assumed.
- Hold one weekly integration/decision meeting. Each pair reports completed deliverables, evidence/tests, experiment changes/results, blockers, and the next dependency.
- Hazem validates pushes and merges accepted work to `develop`. `develop` is the integration line. Hazem merges the final validated result to `master` when the project is ready.
- Do not create a repository, branches, or collaborator access during this document phase. Team GitHub usernames and repository details are still to be supplied in the later setup phase.
- Keep merge decisions traceable: each accepted change should state its purpose, tests/evaluation run, and any contract or dataset impact.

Operating environments

The team has mixed local CPU/GPU computers. Keep a declared CPU-compatible core test and local run path as the reproducible baseline; GPU acceleration is optional and model-dependent. Free cloud plans and Google Colab are optional, not guaranteed or required. Record machine/provider/runtime for each benchmark, and obey the same data permissions in Colab as on local machines. Separate CPU/GPU dependency folders and environment READMEs are part of the later repository setup, not this document phase.

7. AI-agent work and experiment discipline

AI agents can help with implementation and analysis. Each human pair owns correctness and review. Add these rules to the agent instructions during the later repository setup phase.

Safe, reviewable implementation

- Give an agent a bounded task, the current schema/version, expected inputs/outputs, tests, and files it may change. If a label, provider permission, or policy is unknown, the agent must

stop and surface the decision rather than invent it.

- Never provide secrets, unapproved recordings, or identifying data to an agent or external tool. Use synthetic fixtures for routine development.
- Require a human pair review of diffs and test/evaluation evidence before accepting generated code. A successful test does not by itself prove that the test matches the product requirement.
- Keep changes cohesive and testable. “One variable at a time” governs **experimental ablations**; code changes may touch several variables or lines.

Controlled experiments for the later ablation study

For a comparison intended to measure causal impact, change one experimental factor at a time (for example, STT provider, normalization setting, or one fusion weight), while holding the dataset snapshot/split, prompt, other config, and evaluation procedure constant as far as possible. Record:

Required experiment record	What to capture
Run ID and hypothesis	Why the change is being tested and what result would support it.
Single changed factor	Exact old/new value or provider/model version.
Held-constant conditions	Dataset checksum/version and split, other configuration, prompt/schema version, seed where applicable, and runtime environment.
Measures	Per-field metrics, latency, sample counts, failures, and relevant qualitative review.
Artifacts	Commit/branch reference, config, command, output path, and limitations.

Do not infer a winning change from a different dataset split or simultaneous changes to several factors. After isolated comparisons, interactions can be studied as separately labeled experiments.

8. Success criteria

Treat these values from the v1.0 PRD/allocation as hypotheses until EDA and the first baseline confirm them. The old 0.75 “severity” metric is not yet mapped to complaint urgency or customer state; the team must agree which field and metric it measures.

- Complaint-category macro-F1: 0.85 initial hypothesis.
- Old severity macro-F1: 0.75 legacy hypothesis; do not relabel it as complaint-urgency performance until the taxonomy and evaluation protocol are approved.
- Routing agreement with a documented human-reviewed ideal agent: 80% initial hypothesis.
- End-to-end latency: p95 under 10 seconds for clips up to 60 seconds, measured separately by CPU, GPU, or an approved provider configuration.
- Summary coverage: at least 95% of evaluable test clips; manually review faithfulness on 20 clips where available, and report the actual review count if fewer are available.
- Demo reliability: aim for five consecutive clean runs when the final demo flow is stable and the competition format permits.

Always report dataset size, label counts, split policy, model/config, hardware/provider, actual numbers, and limitations. If the data cannot support a metric, state that plainly rather than claiming success.

9. Current open decisions

- Dataset access, data rights, consent, actual labels, and retention period.
- Final category, urgency, and customer-state labels and annotation policy.
- External API/provider permissions and any quota/budget.
- Exact local CPU/GPU hardware and which experiments, if any, use Colab.
- Competition judging criteria, demo format, and deadline.
- Final dashboard language and the named frontend/deployment implementer(s), assigned by Hazem at the end of the core phase before demo-readiness.